Практическая работа

Отчёт по практической работе № 6

Предотвращение инъекций и ошибок работы с файлами

Дисциплина:	Основы разработки защищённого программного обеспечения и компьютерных сетей
Вариант:	19 — Система учёта транспортных средств организации
Исходная версия:	v0.5
Версия работы:	v0.6
Студент:	Мостовая Полина Андреевна
Группа:	ИУСбд-02-23

2026

Содержание

1 Тема, цель и характеристика проекта 2

2 Архитектура 2

3 Безопасный SQLite Repository 2

4 FileStore и политика файлов 3

5 Индивидуальное задание №19: символические ссылки 3

6 Командная инъекция и аргументы процесса 3

7 Таблица тестовых сценариев 3

8 Интеграционная демонстрация 4

9 Ограничения и остаточные риски 4

10 Вывод 5

11 Скриншоты демонстрации и проверок 5

12 Ответы на контрольные вопросы 5

1. Тема, цель и характеристика проекта

Цель работы — безопасно использовать проверенные в ЛР №5 данные при работе с SQLite и файловой системой, исследовать передачу аргументов процессу и проверить компоненты автоматизированными позитивными и негативными тестами [1]. Работа продолжает v0.5. Исходный валидатор проекта называется `validate_request()`, поэтому интеграция сохраняет его фактический API.

Прототип не является веб-сервисом: в нём нет сетевого API, входа пользователей и авторизации заявок. ЛР6 добавляет локальную модель обращения, SQLite-репозиторий, каталог вложений и демонстрацию. Это не означает, что все функции учета автопарка и поездок уже реализованы.

2. Архитектура

На входе `validate_request` ограничивает и проверяет JSON и создаёт неизменяемый `ValidatedRequest`. `RequestService` связывает SQLite и FileStore: сначала сохраняется проверенное вложение, затем строка БД; при ошибке INSERT предпринимается удаление файла.

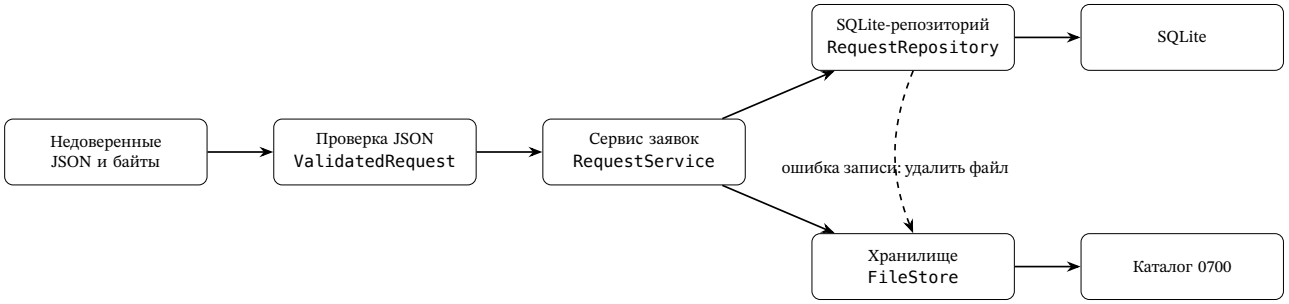


Рис. 1: Компоненты и компенсирующее действие при ошибке SQLite.

3. Безопасный SQLite Repository

`RequestRepository` использует стандартный модуль `sqlite3`. Операции добавления, чтения, изменения приоритета и удаления передают значения через параметры `?`. Сортировка является единственным динамическим SQL-фрагментом и выбирается из констант `newest` и `priority`; пользовательский текст никогда не попадает в него. Маркеры параметров предназначены для значений, а не имён колонок или SQL-операторов [2, 4, 6].

Таблица задаёт первичный ключ, обязательность и диапазоны полей через `NOT NULL/CHECK`; `file_id` уникален. Для пары вложения установлено соответствие: оба поля отсутствуют или оба присутствуют, а ID состоит ровно из 32 строчных шестнадцатеричных символов. `delete_unattached` использует условный `DELETE ... AND file_id IS NULL`, поэтому строка с вложением не удаляется.

4. FileStore и политика файлов

`FileStore` принимает только логический тип вложения и bytes; физический путь от пользователя не принимается. ID формируется через `secrets.token_hex(16)`. Размер ограничен 1 048 576 байт. Создание использует `O_CREAT | O_EXCL`; файл получает права 0600, каталог хранилища — 0700. Чтение ограничено пределом `MAX_BYTES + 1`, проверяет открытый дескриптор через `fstat` и требует обычный файл.

Политика типов: `log` — строгий UTF-8 без NUL; `report` — начальная сигнатура `%PDF-`; `screenshot` и вариант 19 `vehicle_photo` — 8-байтовая PNG-сигнатура. Это первичная фильтрация, не полная проверка PDF/изображения, не антивирусный анализ и не гарантия безопасного рендеринга. JPEG для `vehicle_photo` не поддерживается.

5. Индивидуальное задание №19: символические ссылки

Каталог открывается с `O_DIRECTORY | O_NOFOLLOW`; создание, чтение и удаление выполняются относительно дескриптора каталога через `dir_fd`. Файл открывается с `O_NOFOLLOW`; после открытия тип объекта устанавливается по `fstat`. Такая последовательность избегает отдельной проверки пути перед открытием и уменьшает окно TOCTOU и выхода по traversal-пути [3, 7, 8]. Вызовы и доступность флагов платформозависимы, поэтому при отсутствии необходимых функций хранилище отказывает закрытым образом.

Выполнены три отдельные проверки: (1) ID указывает на ссылку на внешний файл — чтение отклонено; (2) случайно выбранное имя уже является ссылкой — `O_EXCL` не позволяет открыть цель и генератор повторяет попытку; внешний целевой файл не изменяется; (3) корневой каталог после инициализации заменяется ссылкой — операция отказывает. Эти результаты относятся к текущей ОС и не доказывают защиту от администратора ОС, компрометации процесса или небезопасного родительского каталога.

6. Командная инъекция и аргументы процесса

Для CRUD и файловых операций внешняя команда не нужна. Отдельная учебная функция `safe_argument` использует фиксированный исполняемый Python, список аргументов, `shell=False`, таймаут 5 секунд и максимальную длину 200 символов. Тест передает `example; echo TEST`: строка возвращается как один аргумент, команда оболочки не запускается. `shell=False` разделяет команду и аргументы, но программа-цель может трактовать аргумент как опцию или путь; поэтому проверка аргументов всё ещё обязательна [5, 9].

7. Таблица тестовых сценариев

Обязательные 15 ситуаций методички отражены в тестах ниже. Дополнительно тестируются точные типы, ограничение ID, варианты 19 и интеграция. Полный набор проекта на текущем первом проходе: **120 passed, 9 subtests passed** по команде `./.venv/bin/python -m pytest -q`. Повторный запуск полного набора зафиксирован на скриншоте.

Таблица 1: Базовые сценарии методички и текущие результаты

№	Сценарий	Ожидаемый результат / тест	Статус
1	Создание и чтение заявки	Сохранение и чтение полей	Пройдено
2	Изменение приоритета	Обновлённое значение	Пройдено
3	Удаление без вложения	Запись удалена	Пройдено
4	SQL-подобный текст	Сохранён как обычное значение	Пройдено
5	Неизвестная сортировка	Контролируемый отказ	Пройдено
6	Неизвестная категория	Контролируемый отказ	Пройдено
7	Булев приоритет	Отклонён	Пройдено
8	LIMIT вне диапазона	Отклонён	Пройдено
9	Сохранение и чтение файла	Байты совпадают	Пройдено
10	Превышение 1 МиБ	Отклонено	Пройдено
11	Неизвестный тип файла	Отклонён	Пройдено
12	Неверная сигнатура PDF	Отклонена	Пройдено
13	Path Traversal через ID	Отклонён	Пройдено
14	Некорректный UTF-8 log	Отклонён	Пройдено
15	Ошибка SQLite	Файл удалён компенсацией	Пройдено

Вариант 19 проверен тремя тестами (внешняя symlink при чтении; коллизия symlink при создании; замена корня symlink). Отдельно проверено сохранение `vehicle_photo` с PNG-сигнатурой, отказ JPEG-подобной сигнатуры и сквозная запись ключа в SQLite. Автотесты вызываются через `tests/test_v06.py`.

8. Интеграционная демонстрация

Команда `python demo_v06.py` проверяет JSON функцией `validate_request`, создаёт временную SQLite-БД и FileStore, сохраняет заявку с журналом, читает его и сравнивает байты, а затем демонстрирует передачу аргумента. Временный каталог удаляется при завершении. Результат демонстрации зафиксирован на скриншоте вместе с полным запуском тестов.

9. Ограничения и остаточные риски

Компенсационное удаление не образует общей транзакции файловой системы и SQLite; сбой удаления может оставить файл, а аварийное завершение процесса может произойти между двумя шагами. Права каталога и его родителей должны исключать запись посторонних процессов. `O_NOFOLLOW`, `O_DIRECTORY` и `dir_fd` доступны не на каждой ОС; реализация отказывает, если безопасное открытие недоступно. Проверка сигнатуры не разбирает файлы полностью. SQLite-файл является локальным хранилищем без настроенной аутентификации/авторизации и без политики резервирования. Тесты подтверждают только проверенные сценарии текущего прототипа.

10. Вывод

В v0.6 добавлены параметризованный Repository, ограниченный FileStore и сервис, связывающий проверенное обращение с вложением. Вариант 19 проверен отдельными сценариями symlink; ключ `vehicle_photo` из v0.5 сохранён, для него определена PNG-сигнатура. На приложенных снимках показаны вывод демонстрации, полный запуск тестов и три проверки варианта 19.

11. Скриншоты демонстрации и проверок

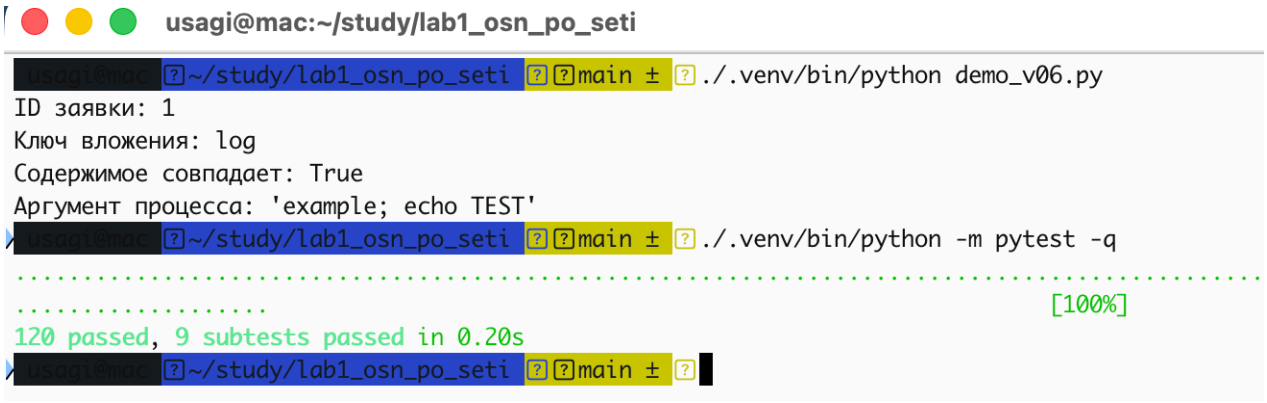


Рис. 2: Вывод сквозной демонстрации и полный запуск тестового набора v0.6.

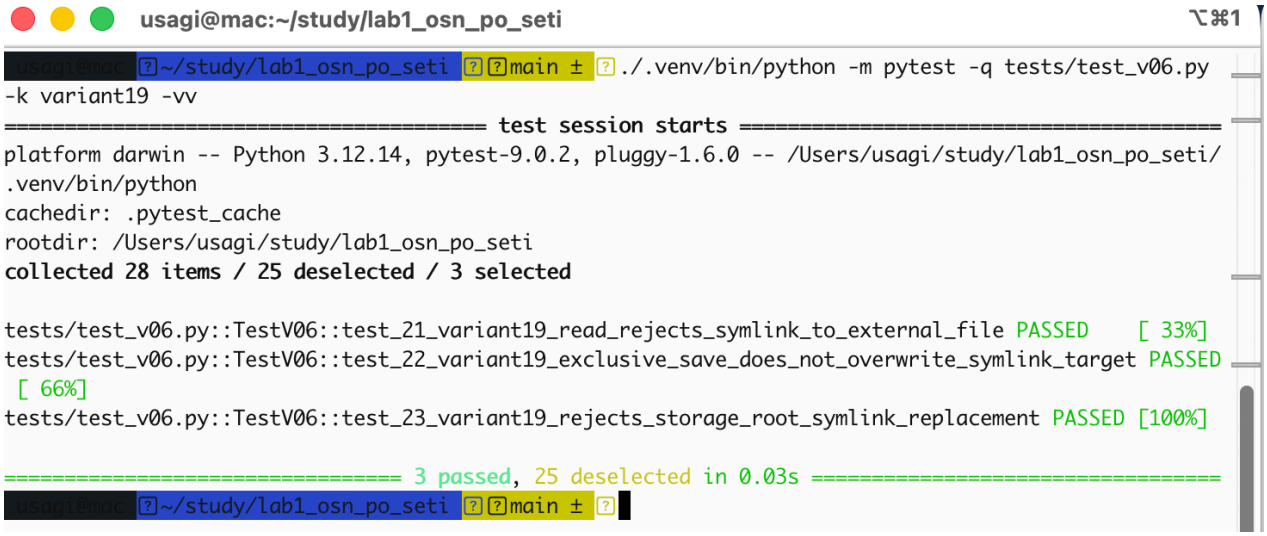


Рис. 3: Три пройденные проверки защиты варианта 19 от symlink.

12. Ответы на контрольные вопросы

1. SQL-инъекция — изменение структуры или смысла SQL внешними данными.
2. Она возникает, когда данные конкатенируют с SQL-текстом вместо передачи отдельно.
3. Параметризованный запрос отделяет команду SQL от значений и связывает их через маркеры.
4. f-строка формирует SQL-текст до вызова драйвера; специальные значения могут превратиться в операторы SQL.

5. Allowlist — фиксированный перечень разрешённых значений, из которого приложение выбирает режим.
6. Repository Layer изолирует операции хранения и их правила от сервисной логики.
7. CRUD — Create, Read, Update, Delete: создать, прочитать, изменить, удалить.
8. Path Traversal — использование пути для выхода за пределы назначенного каталога.
9. Абсолютный путь задаётся от корня файловой системы; относительный — от базового/текущего каталога.
10. TOCTOU — гонка между проверкой ресурса и его использованием, когда состояние ресурса может измениться.
11. `O_EXCL` совместно с `O_CREAT` запрещает создание, если имя уже существует.
12. `O_NOFOLLOW` запрещает следовать по конечному компоненту, если это symlink, на ОС с поддержкой флага.
13. Физическое имя позволяет управлять путём, провоцировать traversal и коллизии; внутренний ID отделяет интерфейс от пути.
14. Лимит ограничивает расход памяти, диска и ресурсов последующей обработки.
15. Сигнатура — лишь ограниченный признак формата; она не подтверждает целостность, безопасную структуру или отсутствие активного содержимого.
16. OS Command Injection — изменение команды или её интерпретации через внешнее значение.
17. При `shell=True` строка разбирается оболочкой; метасимволы могут стать операторами и запустить дополнительные команды.
18. Внедрение аргумента меняет параметры заданной программы; командная инъекция меняет выполняемую команду. Опция может быть опасна и без оболочки.
19. Компенсирующая операция пытается отдельно обратить ранее выполненный шаг после сбоя следующего шага.
20. Файловая система и SQLite не участвуют в общей транзакции; авария или отказ удаления может помешать компенсации.
21. Негативные тесты проверяют, что ошибочные и вредоносно сформированные входы приводят к контролируемому отказу.
22. Успешные тесты подтверждают лишь выбранные сценарии в данном окружении; они не исключают неизвестные дефекты и риски платформы/развертывания.

Список литературы

[1] Методические указания «Практическая работа №6. Предотвращение инъекций и ошибок работы с файлами», предоставленный PDF, 29 страниц.

[2] Python Software Foundation. Документация `sqlite3`: связывание параметров. https://docs.python.org/3/library/sqlite3.html (дата обращения: 29.10.2026).

[3] Python Software Foundation. Документация `os`: `open`, файловые дескрипторы и `dir_fd`. https://docs.python.org/3/library/os.html (дата обращения: 29.10.2026).

[4] OWASP Foundation. SQL Injection Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html (дата обращения: 29.10.2026).

[5] OWASP Foundation. OS Command Injection Defense Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/OS_Command_Injection_Defense_Cheat_Sheet.html (дата обращения: 29.10.2026).

[6] MITRE. CWE-89: Improper Neutralization of Special Elements used in an SQL Command. https://cwe.mitre.org/data/definitions/89.html (дата обращения: 29.10.2026).

[7] MITRE. CWE-22: Improper Limitation of a Pathname to a Restricted Directory. https://cwe.mitre.org/data/definitions/22.html (дата обращения: 29.10.2026).

[8] MITRE. CWE-367: Time-of-check Time-of-use (TOCTOU) Race Condition. https://cwe.mitre.org/data/definitions/367.html (дата обращения: 29.10.2026).

[9] MITRE. CWE-78: Improper Neutralization of Special Elements used in an OS Command. https://cwe.mitre.org/data/definitions/78.html (дата обращения: 29.10.2026).